

第2章 程序基本结构

2.1 程序的基本结构

- C++程序结构：一个C++程序由**注释**、**预编译指令**、**名字空间指令**、**主程序**组成，其中主程序至少要包含 `main` 函数

2.1.1 注释

- 注释与程序：注释在编译过程中会被**忽略**，不会影响程序行为
- 注释的目的：
 - 解释程序中一些比较难以理解的部分
 - 备注信息：作者、修改历史、开源协议等
 - 临时让某些代码无效，便于调试
- 注释的格式：
 - `/*` 和 `*/` 之间的所有文字
 - `//` 到此行结束的所有文字
 - 示例：

```
/* File: main.cpp
 * Term: 2023-2024-1
 * Chapter: 2
 */
```

```
// Print "Hello World!" on the screen
```

2.1.2 预编译指令

- 预编译指令：在编译之前处理的命令
 - 以 `#` 开头，无需分号结尾
- 一类常用的预编译命令为**库包含**
 - 用 `#include` 实现，表示包含某个库
- 库：程序员编写的能够实现特定功能的子程序
 - 每个库要提供接口，即**头文件** `.h`
 - 头文件告诉用户和编译器该库提供了哪些功能，以及如何使用特定功能
 - 以便编译器检查程序中对对应库的调用是否正确
- 库包含的格式：
 - `#include <filename>` 包含系统库

- `#include "filename"` 包含自定义库

2.1.3 名字空间

- 名字空间：把一组实体组合在一起，构成作用域
 - 用于解决重名问题
- C++标准库中的实体定义在名字空间 `std` 中
- 可用 `using namespace std;` 表示使用 `std` 中的实体
 - 这样 `std::cout` 就可以简写为 `cout`

2.1.4 主程序

- 主程序：算法的描述
 - 每个程序都是由一个或多个**函数**组成
 - 其中至少要包含一个 `main` 函数，它是程序的入口
 - `int main()` 称为**函数头**，`{ }` 及其内容称为**函数体**

于是在屏幕上打印"Hello World!"的程序可以写为：

```
/* File: main.cpp
 * Term: 2023-2024-1
 * Chapter: 2
 */

// Print "Hello World!" on the screen

#include <iostream>

using namespace std;

int main()
{
    cout << "Hello World!" << endl;
    return 0;
}
```

2.2 变量与常量

- 变量：编写程序时未确定或程序运行过程中会变化的量
 - 提供具名的、可供程序操作的存储空间
 - 临时存储输入数据、中间结果和最终结果
- 常量：编写程序时已确定且程序运行过程中保持不变的量

2.2.1 变量定义

- 变量3要素：数据类型、名称、值
- 定义格式：<type> <variable 1>, <variable 2>, ..., <variable n>;
 - <type> 为类型名
 - <variable i> 为变量名
- **变量在使用前必须被定义**
- 命名规则：
 - 首字符为**字母**或**下划线**
 - 其他字符为**字母、数字、下划线**之一
 - 不可与系统关键字相同（如 int , while , return 等）
 - 关键字在C++中有特殊用途
 - 区分大小写
 - 标识符的最大长度由编译器指定
 - 标识符应尽量表明变量含义
- 定义位置：任意
 - 推荐在首次使用前定义，防止定义后未使用，造成空间浪费
- 变量初始化：
 - 包括**变量定义、变量赋初值**
 - 变量定义的目的是为变量分配存储空间
 - 变量赋初值的目的是为分配的存储空间指定内容
 - 格式：
 - <type> <variable> = <initial value>;
 - <type> <variable>(<initial value>;
 - 示例：

```
int a = 3;  
char ch('A');
```

- 若变量定义时未赋初值，则变量的值为变量的存储空间上原有的内容
 - 可能会导致程序错误
 - 给变量赋初值是良好的习惯

2.2.2 数据类型

- 分类：**内置数据类型、自定义数据类型**（将在第8章及以后介绍）
- 特征：数据在内存中是如何表示的，也决定了值的范围
 - 数据占据内存空间的大小
 - 数据的布局方式
- **整型：**
 - 分类：

- 按占用内存：
 - 长长整型 long long int 【C++11新增】
 - 长整型 long int
 - 基本整型 int
 - 短整型 short int
- 按符号类型：
 - 有符号：直接用上述关键词
 - 无符号： unsigned +上述关键词
- 占用内存：C++规定了最少内存
 - long：4 Bytes
 - short：2 Bytes
 - int ≥ short
 - 具体由编译器根据操作系统位数指定
- 允许运算：
 - 算术运算： +， -， *， /， %
 - / 为取商的整数部分
 - % 为取余数，称为**模**
 - 比较运算
 - 输入输出运算
- 机内表示：以补码表示
 - 原码：最高位是符号位，其余为数值位
 - 符号位：0表示正数，1表示负数
 - 数值位：二进制表示
 - 反码：正数的反码与原码相同，负数的反码为其绝对值的二进制表示取反
 - 补码：正数的反码与原码相同，负数的反码为其反码+1
- 溢出问题：**整数运算要注意范围**。如1字节的数0111 1111表示127，加上1变为1000 0000表示-128，称为溢出。**C++程序不会检查溢出，需要编程者自己注意**

• 浮点型:

- 机内表示：浮点表示法 $a \cdot 2^b$
 - a 称为尾数， b 称为指数
 - 无法在机内精确表示
- 分类：
 - 单精度 float
 - 双精度 double
 - 长双精度 long double
- 精度：C++规定了最小精度位数：
 - float：6位十进制有效数字
 - double：10位十进制有效数字
- 允许运算：
 - 算术运算： +， -， *， /， 但不允许 %

- 比较运算
 - 输入输出运算
- **字符型 char** :
 - 存放1个字母或符号
 - 占1字节，在机内用字符编码表示
 - 常用的字符编码表是**ASCII**
 - 特性：0~9, A~Z, a~z分别连续、顺序存放
 - 只有0~127有效
 - 允许运算：算术运算、比较运算、输入输出运算
 - ASCII表中部分字符为非打印字符，直接输出不会显示在屏幕上
- **布尔型 bool** : 【C++新增】
 - 占1字节，值为 true 或 false
 - 允许运算：算术运算、比较运算、逻辑运算
 - 进行输入输出运算时不显示 true 或 false ，而是显示 0 或 1
- 求占用空间： sizeof() 函数
 - sizeof(<type>) 表示求该变量类型占据的字节数
 - sizeof(<variable>) 表示求该变量占据的字节数
 - sizeof(<expression>) 表示求该表达式的结果占据的字节数
- 变量别名：给变量类型重新命名
 - 格式： typedef <original name> <current name>;
 - 之后可以用原名或现名定义变量

2.2.3 常量与符号常量

- **整型常量**:
 - 进制表示:
 - 默认十进制，如 123
 - 八进制，如 0123
 - 十六进制，如 0x123
 - 默认标准整型 int
 - 如需看作长整型，可在后面加 L ，如 123L
 - 如需看作无符号数，可在后面加 U ，如 123U
- **浮点数常量**:
 - 十进制表示，如 3.14
 - 科学计数法，如 2e4 表示 2×10^{-4}
 - 尾数不能为空（1 不能省略）
 - 指数必须是整数
 - 一般作 double 处理
 - 如果要看作 float ，可在后面加 F ，如 3.14F
- **字符常量**:
 - 可打印字符：用单引号引起来的1个字符，如 'D'

- 非打印字符：用"转义序列+可打印字符"表示
 - 反斜杠符号 \ 称为转义字符
 - 常用非打印字符：
 - \n 换行符
 - \t 制表符 Tab
 - \h 退格符 Backspace
 - \r 回车符 Enter
 - \f 换页符
 - \' 单引号（仅用于字符常量，如 '\'' 表示字符 ' '）
 - \" 双引号（仅用于字符串常量，如 "\"Hello World!\\"" 表示字符串 "Hello World!"）
 - \\ 反斜杠本身
- 布尔常量：true 和 false
- 符号常量：给程序中的常量取别名
 - 宏定义：【C风格】
 - 格式：#define <constant> <name>
 - 位置：在预编译处
 - 原理：编译器会将 <constant name> 替换为 <name>
 - 不会检查数据类型
 - const 常量：【C++风格】
 - 格式：const <type> <constant name> = <constant value>;
 - 位置：在使用前即可
 - 定义时必须赋初值而且只能在定义处修改该常量的值
 - <constant name> 命名要求与标识符命名要求相同
 - 通常使用全大写字母命名
 - 好处：提高程序可读性；在改变常量时只需修改定义处的值即可一改全改；编译器会检查数据类型

2.3 输入与输出

2.3.1 输入

- 实现：输入流对象 cin 和流提取运算符 >>
- 格式：cin >> <variable 1> >> <variable 2> >> ... >> <variable n>;
- 响应：程序执行到 cin 语句会暂停，等待用户输入，用户输入回车时继续
 - 本质上是 cin 与 <variable 1> 的 >> 运算
 - 用户输入后，程序先读取 <variable 1> 的数据类型，然后跳过空白字符，最后读取变量值
 - 用户输入合法时，cin 与 <variable 1> 的 >> 运算结果为 cin，由程序继续读取，或语句结束；

- 用户输入非法时，`cin` 与 `<variable 1>` 的 `>>` 运算结果为某一输入错误标记，而该输入错误标记与 `<variable 1>` 的 `>>` 运算将什么也不做，即**程序忽略错误处后的语句剩余部分**
- 空白字符处理：`cin.get()` 函数
 - 当期望字符变量 `char` 的输入值为空白字符时，`cin` 和 `>>` 无法实现，因为会跳过空白字符
 - 实现方法：`cin.get(<char>);` 或 `<char> = cin.get();`

2.3.2 输出

- 实现：输出流对象 `cout` 和流插入运算符 `<<`
- 格式：`cout << <param 1> << <param 2> << ... << <param n>;`
 - `<param i>` 可以是变量、常量、表达式，分别打印变量值、常量值、表达式结果
 - `<param i>` 也可以是 `endl`，表示换行

2.4 算术运算

2.4.1 算术表达式

- 作用：完成计算的语句
- 组成：**算术运算符**和**操作数**
- 算术运算符：`+`（加法）`-`（减法）`*`（乘法）`/`（除法，其中整型数据的除法只取商）`%`（模，仅用于整数运算，除法取余数）
- 运算顺序：先算括号里面的，多级括号先算最内层，同层括号先算乘除模后算加减
- **不允许用中括号[]和大括号{}，因为中括号和大括号在C++中有其他用途**
 - **多层括号只能用多层小括号()嵌套**
- 结合性：左结合，即**同级运算从左到右**
- 运算对象：整型、浮点型、字符型、布尔型（变量或常量）

2.4.2 不同类型数据的运算

- 机理：C++程序只能执行同类型数据的运算，且运算结果与运算数的数据类型相同
 - 在执行不同类型数据的运算时要先转化为同一数据类型
- 自动转换：由编译器完成的数据类型转换
 - 字符型、布尔型转化为整型
 - 较短的整型转化为较长的整型
 - 整型转化为浮点数
 - 较粗略的浮点数转化为较精确的浮点数
- 自动转换原则：
 - 占用空间少的向占用空间多的转换
 - 数值范围小的向数值范围大的转换

2.4.3 强制类型转换

- 格式：
 - `(<type>)(<expression>)`
 - `<type>(<expression>)`
- 强制类型转换可能产生错误，应尽量避免使用

2.4.4 数学函数库

- 除 $+$, $-$, $*$, $/$, $\%$ 之外的数学运算需要通过函数实现
 - `<cmath>` 库提供了部分函数
 - 可通过预编译指令包含该库，此后就可在主程序中使用库函数：
 - `#include <math.h>` (C风格库包含)
 - `#include <cmath>` (C++风格库包含)
- 主要函数：(默认操作数和结果都是 `double`)
 - 绝对值 $|x|$:
 - `abs(x)` , x 为 `int` 时使用
 - `fabs(x)` , x 为 `double` 时使用
 - 指数 e^x : `exp(x)`
 - 幂运算 x^y : `pow(x, y)`
 - 平方根 $\sqrt{x}$: `sqrt(x)`
 - 对数：
 - $\ln x$: `log(x)`
 - $\lg x$: `log10(x)`
 - 三角函数 $\sin x, \cos x, \tan x$: 分别为 `sin(x)` , `cos(x)` , `tan(x)`
 - 反三角函数 $\arcsin x, \arccos x, \arctan x$: 分别为 `asin(x)` , `acos(x)` , `atan(x)`

2.5 赋值运算

2.5.1 赋值表达式

- 格式： `<variable> = <expression>;`
- 原理：
 - i. 计算 `<expression>` 的值
 - ii. 将结果存入 `<variable>`
 - iii. 用**变量的值**替换程序中的 `<variable> = <expression>` 并继续执行
- **程序语言与代数式的区别**：在代数中 $x = x + 1$ 表示一个无解的方程，而在C++语言中 `x = x + 1` 表示取出变量 `x` 中存放的值，将其 $+1$, 之后将结果存入变量 `x`
- 左值与右值：
 - 左值：能出现在赋值运算符左边或右边的值
 - 右值：只能出现在赋值运算符左边的值

- **左值必有内存空间**
- 类型转换：根据**变量的数据类型**，强制转换表达式结果的数据类型
 - float 和 double 取整后转换为 int
 - int 数值不变转换为 float 和 double
 - char 取机内码转换为 int
 - int 取机内表示的最低8位转换为 char
 - bool 取 true =1， false =0转换为 int
 - int 为0转换为 false ，不为0转换为 true

2.5.2 多重赋值

- 赋值运算是右结合的，即**同级运算从右到左**
 - 因此可在统一赋值表达式中同时给多个左值赋值
 - 优点：方便，代码量小
 - 缺点：降低了可读性；可能产生难以预料的错误

考察如下代码段：

```
double d;  
int i;  
d = i = 1.5;
```

根据运算原理，1.5 转换为 1 后存入变量 i ，然后用 1 替换 i = 1.5 ，则后面执行的是 d = 1 ， 1 转换为 1.0 后存入变量 d ，与预期得到的 1.5 不符

2.5.3 赋值与算术的复合

- 为使代码更加简洁可读，在C/C++中以下2个表达式等效：
 - <variable i> = <variable i> <operator> <expression>;
 - <variable> <operator>= <expression>;
- 如 x = x + 1; 可简写为 x += 1;

2.5.4 自增、自减运算符

- 格式：
 - 前缀表达式： <variable>++ ， <variable>--
 - 后缀表达式： ++<variable> ， --<variable>
- 原理：
 - ++ 使变量 +1 ， -- 使变量 -1
 - 前缀表达式的结果为变量**自增或自减之前**的值，后缀表达式结果为变量**自增或自减之后**的值

考察如下代码段：

```
int i = 2;  
int j = ++i;  
int m = 2;  
int n = m--;
```

则：最终 i =3, j =3, m =1, n =2